NEXUS

Project Management SaaS

PRODUCT REQUIREMENTS DOCUMENT

React + TypeScript • Django REST Framework • PostgreSQL

Version 1.0 | MVP planning specification | 27 October 2026

Prepared for portfolio development and Upwork project showcasing

Document Guide

1. Executive Summary

2. Product Vision, Goals & Non-Goals

3. Users & Roles

4. Core User Journeys

5. Information Architecture & Pages

6. Functional Requirements

7. Non-Functional Requirements

8. Recommended Technical Architecture

9. Database Recommendation & Deployment Fit

- Document PythonAnywhere reload/redeploy steps, migration commands, environment configuration, logs/troubleshooting, and any manual steps required by the free plan in docs/DEPLOYMENT.md.
- A deployment is accepted only when the production frontend can call the production API over HTTPS, Django can connect to Neon, migrations complete, core user journeys pass, static assets load and authorization tests pass.
- Provide a lightweight health endpoint that reports application readiness without exposing credentials, internal errors or database details. Where practical, verify database reachability safely.

Health checks and deployment acceptance

7. Verify end to end: test registration/login, workspace and project creation, task creation and status changes, comments/activity, refresh persistence, permissions between workspaces, CORS/auth behavior, static assets, error states and API response times. Confirm migrations target the intended Neon database and that no secrets appear in Git or client bundles.

6. Deploy on Vercel: configure frontend/ as the project root; set VITE_API_BASE_URL to the actual PythonAnywhere API base URL in the relevant environment; ensure the API client reads that variable rather than hardcoding localhost; deploy and verify browser requests. Never expose DATABASE_URL, DJANGO_SECRET_KEY or other server secrets in frontend variables.

5. Deploy on PythonAnywhere: deploy the repository using a method supported by the selected plan; create a virtual environment using a supported Python version; install requirements; configure the WSGI file to import the actual Django project; configure secrets securely; run system checks and migrations; run collectstatic if required; configure static-file serving; reload the web app; and test the health endpoint and API. Use actual module paths from the implementation, not invented placeholders.

4. Configure production Django settings: DJANGO_DEBUG=False; a unique DJANGO_SECRET_KEY; DJANGO_ALLOWED_HOSTS set to the real PythonAnywhere host; DATABASE_URL set to the Neon connection string; CORS_ALLOWED_ORIGINS limited to the real Vercel frontend origin; and CSRF_TRUSTED_ORIGINS/cookie settings appropriate to the selected auth flow. Keep local values in .env.example only.

3. Configure Neon: create the PostgreSQL project/database, obtain its connection string, use SSL as required, and store credentials in backend-only secure configuration. Use the correct PostgreSQL driver and Django database settings. Consider Neon pooling only if appropriate for the connection pattern and available plan.

2. Prepare the backend: pin dependencies in backend/requirements.txt; confirm manage.py, the Django settings module, URL configuration and WSGI entry point; run tests and Django checks locally. Keep the existing modular Django monolith and PostgreSQL data model.

1. Preflight compatibility: check PythonAnywhere's current free-account rules, supported Python version, web-app/WSGI configuration, package installation, static-file support, outbound networking and usage restrictions. Confirm a test connection from PythonAnywhere to Neon before treating the target architecture as viable.

PythonAnywhere deployment runbook

10. ERD — Entity Relationship Design

11. API Contract (REST)

12. Git Branching & Collaboration Strategy

13. Suggested Implementation Milestones

14. Testing & Definition of Done

15. Security & Privacy Checklist

16. Risks & Mitigations

17. Portfolio & GitHub Deliverables

18. Recommended Build Order & Immediate Next Steps

1. Executive Summary

Nexus is a portfolio-grade, multi-tenant project management SaaS application inspired by modern tools such as Linear and Jira. It enables people to create workspaces, invite teammates, organize projects, manage tasks on a Kanban board, discuss work, and understand progress through dashboards and activity history.

The product is intended to demonstrate production-minded full-stack engineering: React UI development, Django REST APIs, relational data modeling, authentication and authorization, tenant isolation, testing, deployment, and a documented Git workflow.

Product principle: deliver a small, coherent, reliable product before adding advanced features. The MVP must work end-to-end in the deployed demo.

Item	Decision
Working name	Nexus
Product type	Multi-tenant project management web application
Primary users	Workspace owners, admins, project managers, members
Frontend	React + Vite + TypeScript
Backend	Python + Django + Django REST Framework
Database	PostgreSQL; Neon managed PostgreSQL free tier, subject to current limits and connectivity verification
Frontend hosting	Vercel free tier for React/Vite frontend; verify current plan limits before deployment
Backend hosting	PythonAnywhere free tier for Django/DRF backend, conditional on verification of outbound Neon PostgreSQL connectivity and required web-app features
Repository	GitHub monorepo with frontend/ and backend/ directories
MVP priority	Authentication, workspaces, projects, tasks, Kanban, comments, activity feed, dashboard

2. Product Vision, Goals & Non-Goals

Vision

Give small teams a clear, low-friction place to plan projects, assign work, discuss tasks, and see what is progressing or blocked.

Goals

- Provide a polished, responsive user experience that is easy to demonstrate to Upwork clients.
- Support multiple workspaces and isolate each workspace's data securely.

- Allow teams to manage projects and tasks using both a list and Kanban workflow.
- Provide task ownership, due dates, priority, comments, and a basic activity timeline.
- Ship a live demo with seeded sample data and a safe demo account.
- Show professional engineering practices through tests, CI, documentation, environment separation, and pull requests.

Non-goals for the first release

- Native iOS or Android apps.
- Enterprise SSO, SCIM, advanced audit compliance, or complex billing.
- Full Slack/email integrations, AI task planning, or advanced automation rules.
- Offline-first editing or conflict-free collaborative editing.
- Microservices; the initial backend will be a modular Django monolith.
- Real-time WebSockets in the first MVP; polling or manual refresh is sufficient for notifications/activity unless time remains.

3. Users & Roles

Role	Capabilities
Workspace Owner	Owns workspace; updates workspace settings; manages members and roles; can archive/delete workspace subject to safeguards.
Workspace Admin	Manages workspace details and members; can create and manage projects; cannot transfer ownership unless explicitly granted.
Project Manager	Creates/updates projects and tasks within permitted workspaces; assigns work; manages project workflow.
Member	Views permitted projects; creates and updates tasks according to project permissions; comments and changes task status.
Viewer (optional MVP extension)	Read-only access to workspace/project content.

Authorization must be enforced by the backend for every protected API request. Hiding buttons in React is not security. All workspace-scoped records must be queried through a verified workspace membership or project access check.

4. Core User Journeys

1. Visitor opens the landing page, reviews product features, and chooses Sign up or Log in.
2. User creates an account, verifies/activates it according to the chosen MVP email strategy, and signs in.
3. First-time user creates a workspace and becomes its owner.
4. Owner creates a project, selects a workflow/status set, and invites or adds teammates.
5. Member opens the project board, creates a task, assigns it, sets priority and due date, and moves it between statuses.
6. Teammates comment on the task; significant actions appear in the activity feed.
7. Workspace member views dashboard metrics, filters tasks, and opens task details.
8. User signs out; refresh/session behavior is handled safely and predictably.

5. Information Architecture & Pages

Route / page	Purpose & main elements	Access
/ (Landing)	Product value proposition, feature highlights, screenshots, CTA buttons, footer.	Public
/login	Email/password form, validation, password visibility, link to registration.	Public
/signup	Name, email, password, confirm password, terms acknowledgement if needed.	Public
/forgot-password	Request password reset; show generic success message to avoid account enumeration.	Public
/reset-password/:uid/:token	Set a new password using a time-limited token.	Public with valid token
/app (redirect)	Redirect to last workspace or workspace picker.	Authenticated
/app/workspaces/new	Create workspace with name and optional slug.	Authenticated
/app/workspaces	List workspaces available to the current user; create/switch workspace.	Authenticated
/app/:workspaceSlug/dashboard	Overview cards, tasks due soon, overdue tasks, recent activity, project summary.	Workspace member
/app/:workspaceSlug/projects	Project list/grid, search, status filter, create project.	Workspace member
/app/:workspaceSlug/projects/new	Create project: name, description, key, dates, status, optional color.	Allowed role
/app/:workspaceSlug/ projects/:projectKey/board	Kanban columns, draggable task cards, filters, create task, quick status changes.	Project access
/app/:workspaceSlug/ projects/:projectKey/list	Table/list view with sorting, pagination, filters and column values.	Project access
/app/:workspaceSlug/tasks/:taskId	Task detail drawer or page: title, description, status, priority, assignee, reporter, dates, comments, activity.	Task access
/app/:workspaceSlug/my-tasks	Tasks assigned to current user across accessible projects; status and due-date filters.	Workspace member
/app/:workspaceSlug/activity	Chronological workspace activity feed with actor, action, target and timestamp.	Workspace member
/app/:workspaceSlug/members	Member list, invite/add member, role management, remove member with confirmation.	Admin/owner for mutations
/app/:workspaceSlug/settings	Workspace name, slug, preferences, archive/delete safeguards.	Admin/owner
/app/profile	Edit display name/avatar placeholder, change password, sign out.	Authenticated

/404	Friendly not-found page and route back to app.	Public

MVP note: invitations can initially be implemented as owner/admin adding an existing registered user by email, with a clear error if no account exists. Email invitations can be added later when an email provider and delivery settings are configured.

6. Functional Requirements

6.1 Authentication & account

- Users can sign up, log in, log out, and request a password reset.
- Validate email uniqueness, password policy, required fields, and server-side errors.
- Use Django's built-in user model or a custom user model created at project start; do not change user model late in development.
- JWT access token should be short-lived; refresh token handling must follow the selected library's documented secure pattern.
- Prefer refresh token in an HttpOnly, Secure cookie in production, with appropriate SameSite and CSRF protections. If using Authorization headers for both tokens, document XSS risks and avoid persistent localStorage storage where possible.
- Authenticated API endpoints return 401 when credentials are absent/invalid; frontend handles expiry and re-login cleanly.
- Rate-limit sensitive authentication endpoints where feasible.

6.2 Workspaces & membership

- A user can belong to multiple workspaces.
- The creator becomes the workspace owner.
- Workspace slugs are unique; changing a slug must not silently break data relationships.
- Members have a role and membership status; only active members can access workspace data.
- Admins/owners can add existing users by email and change roles according to role policy.
- Prevent removing the final owner; define ownership transfer or block destructive action.
- All queries and mutations enforce workspace membership and tenant isolation.

6.3 Projects

- Create, read, update, archive and (with confirmation) delete projects.
- Each project belongs to exactly one workspace and has a unique human-readable key within that workspace.
- Project fields: name, key, description, status, optional start/due dates, creator, timestamps.
- Project list supports search, status filtering, pagination and sorting.
- Only authorized workspace members can access a project; role-specific mutations are enforced on backend.

6.4 Tasks & Kanban

- Create, view, update, archive/delete tasks according to permissions.
- Task fields: title, description, status, priority, assignee, reporter/creator, due date, position/order, project, timestamps.

- Tasks support statuses such as Backlog, To Do, In Progress, In Review, Done; status configuration can be fixed for MVP.
- Tasks support priorities: Low, Medium, High, Urgent.
- Drag-and-drop updates task status and ordering; server validates destination status and project access.
- Task cards show title, task key/ID, priority, assignee avatar/initials, and due-date indicator.
- Filters include status, priority, assignee, due-date state and text search.
- Use pagination for task lists and define sensible limits for board queries.

6.5 Comments & activity

- Members with task access can add comments; authors can edit/delete their own comments, while admins' moderation rules should be explicitly defined.
- Activity entries are created for important actions such as task creation, status changes, assignment changes, comments, and project updates.
- Activity records include actor, workspace, optional project/task target, event type, concise metadata, and timestamp.
- Activity feed is paginated and sorted newest first.
- Do not store secrets or unnecessary personal data in activity metadata.

6.6 Dashboard & notifications

- Dashboard shows active project count, tasks assigned to the current user, overdue tasks, tasks due soon, and completed tasks in a selected period.
- Metrics must be derived from authorized workspace data and have clearly defined date boundaries/time zone.
- Basic in-app notifications may be added for assignment and comment events; if scope is tight, ship the activity feed first.
- Notification read/unread state is per recipient and does not expose another user's private notification records.

6.7 Search, errors & accessibility

- Search and filters are reflected in URL query parameters where practical, allowing reload/share of views.
- Show loading, empty, success, and error states for all primary data flows.
- Use confirmation for destructive actions and optimistic UI only when rollback/error handling is implemented.
- Support keyboard focus, visible focus indicators, accessible labels, contrast, and keyboard operation for core workflows.
- Use consistent toast/inline error patterns; never show raw stack traces to users.

7. Non-Functional Requirements

Area	Requirement
Security	Server-side authorization on every endpoint; tenant isolation; password hashing via Django; secrets in environment variables; HTTPS in production.
Performance	Paginated APIs; avoid N+1 queries using select_related/prefetch_related; add indexes for common filters; target a responsive interface under normal demo load.

Reliability	Database constraints, transactions for multi-step mutations, consistent error response format, migrations checked into Git.
Maintainability	Feature-based frontend structure, modular Django apps, lint/format tools, documented naming conventions.
Accessibility	Semantic HTML, keyboard-friendly controls, labels, focus states, reasonable color contrast.
Responsive UI	Support mobile, tablet, laptop and desktop; Kanban may use horizontal scrolling on small screens.
Observability	Structured server logs; production error tracking can be added if a free tier is suitable; never log passwords or tokens.
Privacy	Use synthetic demo data; provide account deletion/data cleanup path appropriate for the demo; avoid real customer data.
Browser support	Current Chrome, Edge, Firefox and Safari; test at least Chromium desktop and mobile-sized viewport.

8. Recommended Technical Architecture

Use a GitHub monorepo with independently deployable frontend and backend. Keep the backend a modular Django monolith; do not introduce microservices for a portfolio MVP.

```
nexus/
├── frontend/              # React + Vite + TypeScript
│   ├── public/
│   ├── src/
│   │   ├── app/           # router, providers, app setup
│   │   ├── components/    # shared UI components
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── workspaces/
│   │   │   ├── projects/
│   │   │   ├── tasks/
│   │   │   ├── comments/
│   │   │   ├── activity/
│   │   │   └── dashboard/
│   │   ├── lib/           # API client, utilities
│   │   ├── hooks/
│   │   ├── types/
│   │   └── styles/
│   ├── tests/
│   ├── .env.example
│   └── package.json
├── backend/
│   ├── config/            # Django settings, URLs, WSGI/ASGI
│   ├── apps/
│   │   ├── accounts/
│   │   ├── workspaces/
│   │   ├── projects/
│   │   ├── tasks/
│   │   ├── comments/
│   │   ├── activity/
```

```
│   │   └── notifications/     # optional MVP extension
│   ├── tests/
│   ├── manage.py
│   ├── requirements.txt
│   └── .env.example
├── docs/
│   ├── PRD.md
│   ├── ERD.md
│   ├── API.md
│   ├── DEPLOYMENT.md
│   └── CONTRIBUTING.md
├── .github/workflows/        # CI workflows
├── .gitignore
└── README.md
```

Frontend recommendations

- React + Vite + TypeScript.
- React Router for routing.
- TanStack Query for server-state fetching, caching and invalidation.
- React Hook Form + Zod for form handling and client validation; repeat validation on backend.
- Tailwind CSS + shadcn/ui for accessible, consistent UI primitives.
- A maintained drag-and-drop library such as dnd-kit for Kanban interactions.
- Recharts for dashboard charts if charts are included.
- ESLint + Prettier; Vitest + React Testing Library for component/unit tests; Playwright for end-to-end smoke tests.

Backend recommendations

- Python 3.11+ (use a supported Python version available on the chosen host), Django + Django REST Framework.
- PostgreSQL as the only primary database.
- Django ORM and migrations; avoid raw SQL unless justified and reviewed.
- django-filter for list filtering; drf-spectacular or equivalent for OpenAPI schema/docs.
- JWT library compatible with the chosen DRF version; configure cookie/CORS/CSRF strategy deliberately.
- pytest + pytest-django for tests; Ruff for linting/formatting if desired.
- Use Django's standard password hashing and validation; do not implement cryptography yourself.

9. Database Recommendation & Deployment Fit

Recommended database: PostgreSQL

Use PostgreSQL because the product contains relational entities and needs consistent relationships among workspaces, memberships, projects, tasks, comments and activity. PostgreSQL supports foreign keys, transactions, unique constraints, indexing, and robust reporting queries.

Hosting pattern

9. Frontend: deploy the React/Vite app to Vercel. In the monorepo, set the project root to frontend/ and configure VITE_API_BASE_URL with the deployed PythonAnywhere API URL.
10. Backend: deploy Django REST Framework to PythonAnywhere as the preferred target. Before implementation depends on it, verify that the selected free account supports the required Python version,

WSGI configuration, outbound HTTPS/database networking, dependency installation, static-file setup and usage limits. If Neon connectivity is blocked, treat that as a deployment blocker and select a compatible Python host rather than silently changing the database or framework.

11. Database: use Neon managed PostgreSQL. Store the Neon connection string only in backend-side configuration, enable SSL as required by Neon, and confirm PythonAnywhere can reach the database. Use separate development and production credentials/databases where plan limits allow.
12. Configure production secrets through a secure configuration method supported by PythonAnywhere and Vercel. Never commit .env files, private keys, database URLs or production credentials. Only browser-safe variables may use the VITE_ prefix.
13. Configure Django ALLOWED_HOSTS for the actual PythonAnywhere hostname. Restrict CORS_ALLOWED_ORIGINS to the actual Vercel frontend origin and any explicitly required local development origin. Configure CSRF_TRUSTED_ORIGINS, cookie settings and token/session behavior to match the selected authentication strategy; CORS is not a CSRF defense.
14. Deploy in this order: create Neon database; deploy/configure Django on PythonAnywhere; apply migrations and create fictional seed data/demo account; verify health and API endpoints; deploy Vercel frontend with its production API base URL; then test end-to-end workflows.
15. Do not assume PythonAnywhere free-tier support for unrestricted outbound connections, custom domains, scheduled tasks, persistent processes or all deployment features. Verify current plan rules and test Neon connectivity before committing. Document any usage limits, request restrictions or latency in the portfolio demo.

> Free-tier reality check: PythonAnywhere, Vercel and Neon plan features and limits can change. Verify current official limits before deployment, especially PythonAnywhere outbound database connectivity to Neon, supported WSGI/Python setup, package and static-file requirements, and usage limits. Vercel hosts the frontend; PythonAnywhere runs Django; Neon stores PostgreSQL data. Free plans may have limited availability, throughput, backups or support and are suitable for a portfolio demo, not a production uptime guarantee.

Environment variable examples

```
# frontend/.env.example
VITE_API_BASE_URL=http://localhost:8000/api

# backend/.env.example
DJANGO_SECRET_KEY=replace-me
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
DATABASE_URL=postgresql://user:password@localhost:5432/nexus
CORS_ALLOWED_ORIGINS=http://localhost:5173
CSRF_TRUSTED_ORIGINS=http://localhost:5173
# Configure JWT/cookie settings according to the selected auth approach
```

The examples above are local-development templates only. In production, set DJANGO_DEBUG=False, use a unique strong secret, use the real PythonAnywhere hostname in DJANGO_ALLOWED_HOSTS, use the deployed Vercel origin for CORS, and store the actual Neon DATABASE_URL only in secure backend configuration. PythonAnywhere's supported configuration mechanism may differ from a typical container host; follow its current documentation and never commit a secrets file.

10. ERD — Entity Relationship Design

The MVP data model below is normalized for PostgreSQL. Use UUID primary keys for public-facing entities where practical; use Django's user model for account identity. The exact implementation can use integer IDs internally if preferred, but avoid exposing sequential IDs as security controls.

```
User (Django AUTH_USER_MODEL)
  1 ———< WorkspaceMembership >——— 1 Workspace
  1 ———< Workspace (created_by)
Workspace
  1 ———< Project
  1 ———< WorkspaceMembership
  1 ———< ActivityEvent
Project
  1 ———< Task
  1 ———< ProjectMembership (optional; omit if workspace-level access only)
  1 ———< ActivityEvent
Task
  0..1 —— assigned_to User
  1 ———< Comment
  1 ———< ActivityEvent
User
  1 ———< Comment (author)
  1 ———< ActivityEvent (actor)
  1 ———< Notification (recipient, optional MVP extension)
```

Entities & key fields

Entity	Important fields / constraints
User	Use Django AUTH_USER_MODEL: id, username/email, password hash, first_name, last_name, is_active, date_joined. Prefer unique normalized email if email is the login identifier.
Workspace	id UUID, name, slug unique, created_by FK User, created_at, updated_at, archived_at nullable.
WorkspaceMembership	id, workspace FK, user FK, role enum (OWNER/ADMIN/MANAGER/MEMBER/VIEWER), status (ACTIVE/INVITED/REMOVED), joined_at, created_at; UNIQUE(workspace_id, user_id).
Project	id UUID, workspace FK, name, key, description, status, start_date nullable, due_date nullable, created_by FK User, created_at, updated_at, archived_at nullable; UNIQUE(workspace_id, key).
Task	id UUID, project FK, title, description, status, priority, assignee FK User nullable, created_by FK User, due_date nullable, position numeric/integer, created_at, updated_at, completed_at nullable, archived_at nullable.
Comment	id UUID, task FK, author FK User, body, created_at, updated_at, deleted_at nullable (soft delete optional).
ActivityEvent	id UUID, workspace FK, actor FK User nullable, project FK nullable, task FK nullable, event_type, metadata JSONB, created_at. Add indexes on workspace_id + created_at and task_id + created_at.
Notification (optional)	id UUID, recipient FK User, workspace FK, activity FK nullable,

	type, read_at nullable, created_at. Index recipient_id + read_at + created_at.

Relationship rules

- A workspace has many memberships; a user can have many workspace memberships.
- A workspace has many projects; a project belongs to one workspace.
- A project has many tasks; each task belongs to one project.
- A task may be assigned to one user; an unassigned task is allowed.
- A task has many comments; each comment has one author.
- Activity events belong to a workspace and can reference a project/task when relevant.
- An assignee must be an active member of the same workspace as the task's project; validate this in backend service/serializer logic.
- Keep foreign-key deletion behavior deliberate. Prefer PROTECT or soft deletion for core business records and CASCADE only where data loss is expected.

Optional project-level membership is not required for MVP if all active workspace members can see all projects. Add ProjectMembership only if the product needs project-specific access controls; do not add complexity without a clear requirement.

11. API Contract (REST)

Prefix all endpoints with /api/v1/. Use JSON requests/responses and consistent status codes. Exact route naming may be adjusted during implementation, but document the final contract in OpenAPI.

Method + endpoint	Purpose / authorization
POST /auth/register/	Create account; public; validate and throttle.
POST /auth/token/	Obtain access/refresh credentials; public; throttle.
POST /auth/token/refresh/	Refresh access token according to auth strategy.
POST /auth/logout/	Invalidate/revoke refresh token if supported by chosen implementation.
POST /auth/password-reset/	Request reset email; generic response.
GET/PATCH /users/me/	Read/update current user's profile.
GET/POST /workspaces/	List user's workspaces / create workspace.
GET/PATCH/DELETE /workspaces/{id}/	Read/update/archive workspace; strict owner/admin permissions.
GET/POST /workspaces/{id}/members/	List/add members; admin/owner for mutations.
PATCH/DELETE /workspaces/{id}/members/{membershipId}/	Change role or remove member; protect final owner.
GET/POST /workspaces/{id}/projects/	List/create projects with pagination, search and filters.
GET/PATCH/DELETE /projects/{id}/	Read/update/archive project; authorized workspace role.
GET/POST /projects/{id}/tasks/	List/create project tasks.
GET/PATCH/DELETE /tasks/{id}/	Read/update/archive task; check workspace membership.
PATCH /tasks/{id}/position/	Change task status/order after validating destination.

GET/POST /tasks/{id}/comments/	List/create comments for task.
PATCH/DELETE /comments/{id}/	Edit/delete comment according to author/admin rules.
GET /workspaces/{id}/activity/	Paginated workspace activity feed.
GET /workspaces/{id}/dashboard/	Authorized aggregate counts and due-soon/overdue summaries.
GET /workspaces/{id}/my-tasks/	Current user's assigned tasks with filters.

API standards

- Return 200 for successful reads/updates, 201 for creation, 204 for successful deletion with no body, 400 for validation errors, 401 unauthenticated, 403 forbidden, 404 not found, and 429 when throttled.
- Paginated list response should have a consistent shape, e.g. count, next, previous, results.
- Use server-side validation for enum values, due dates, membership, status transitions and field lengths.
- Do not leak whether a private workspace/project exists to an unauthorized user; use a consistent 404/403 policy.
- Document sample payloads, error responses, query parameters and authentication in API.md/OpenAPI.

12. Git Branching & Collaboration Strategy

Use short-lived feature branches and pull requests, even if you are the only developer. This demonstrates a realistic team workflow and keeps main deployable.

Branch naming

Branch	Purpose
main	Stable, deployable code; protected where possible; production deployments originate here.
develop (optional)	Integration branch for a multi-feature release. For a solo MVP, omit it if it creates unnecessary merge overhead.
feature/auth-registration	One feature or vertical slice.
feature/workspace-management	Workspace and membership behavior.
feature/project-crud	Project creation and listing.
feature/task-kanban	Task APIs and Kanban UI.
feature/task-comments	Comments and activity events.
feature/dashboard-analytics	Dashboard metrics and widgets.
fix/tenant-isolation	Bug/security fix.
test/api-permissions	Test-only or test-focused work.
docs/deployment-guide	Documentation updates.
chore/ci-linting	Tooling, dependencies, CI and maintenance.
release/v1.0.0 (optional)	Release preparation if a release branch is useful.

Recommended solo workflow

16. Start from the latest main: git switch main && git pull.
17. Create a small branch: git switch -c feature/workspace-management.
18. Make focused commits with clear messages, e.g. feat(workspaces): add workspace creation API.
19. Run relevant backend/frontend tests, lint and type checks locally.
20. Push the branch: git push -u origin feature/workspace-management.
21. Open a GitHub pull request into main (or develop if you adopt it); describe scope, screenshots, tests, migrations and any environment changes.
22. Review your own diff, verify no secrets or generated files are committed, and check CI.
23. Merge the PR, delete the feature branch, pull latest main, and start the next branch.
24. Deploy automatically from main only after checks pass; verify the deployed smoke test.

Commit message convention

```
feat(auth): add registration endpoint
feat(tasks): support task priority and due dates
fix(workspaces): enforce tenant membership checks
test(api): cover project permission rules
docs(readme): add local setup instructions
chore(ci): run frontend lint and backend tests
```

Git safety rules

- Never commit .env, API keys, JWT secrets, database credentials, private dumps, or real customer data.
- Commit migrations with the model changes that require them.
- Do not commit node_modules, virtual environments, build outputs, local SQLite databases or coverage artifacts.
- Use .env.example files containing placeholder values only.
- Do not force-push main; avoid rewriting shared history.
- Keep PRs focused and small enough to review.

13. Suggested Implementation Milestones

Milestone	Deliverables	Acceptance criteria
M0 — Foundation	Monorepo, React/Vite/TS setup, Django/DRF setup, PostgreSQL connection, formatting/linting, CI skeleton, base README.	Frontend and backend run locally; health endpoint responds; CI runs basic checks.
M1 — Auth & tenant foundation	User model, signup/login/logout/refresh, protected routes, workspace creation, membership model and permission helpers.	User can register/login/create workspace; cross-workspace access tests pass.
M2 — Projects	Project CRUD, workspace project list, search/filter/pagination, project details.	Authorized members can manage projects; unauthorized users cannot access them.
M3 — Tasks & board	Task CRUD, status/priority/assignee/due date, list view, Kanban drag-and-drop and ordering.	Task changes persist after refresh; invalid transitions/assignees are rejected.
M4 — Collaboration	Comments, activity feed, task detail page, basic notifications if time permits.	Comment and important task changes appear in the correct timeline.

M5 — Dashboard & polish	Dashboard metrics, empty/loading/error states, responsive design, accessibility, sample data.	Dashboard totals match database queries; core pages work on mobile and desktop.
M6 — Quality & security	Backend permission tests, frontend tests, E2E smoke tests, validation, rate limiting, dependency checks.	Critical flows pass; no known high-severity authorization issue remains.
M7 — Deployment & portfolio	Neon PostgreSQL, PythonAnywhere Django backend, Vercel frontend, environment configuration, demo account, screenshots, demo video and deployment docs.	Live demo can be used by a client; frontend-to-API-to-database flow works; no secret is exposed; setup and redeployment are documented and reproducible.

Build vertical slices: for example, complete workspace API + React page + tests before moving on. Avoid writing the entire backend first and leaving the UI until the end.

14. Testing & Definition of Done

Required test coverage areas

- Authentication: valid/invalid credentials, duplicate email, expired token, logout/refresh behavior.
- Tenant isolation: user from Workspace A cannot list, read, edit or delete Workspace B records by changing IDs.
- Role permissions: member cannot perform owner-only member management; final owner cannot be removed.
- Projects: key uniqueness within workspace, invalid date ranges if constrained, unauthorized access.
- Tasks: valid status/priority, assignee membership, ordering updates, due-date filtering, task access.
- Comments/activity: only authorized users can comment; activity is emitted for relevant actions.
- Frontend: login validation, protected route, project creation, task card status changes, loading/error states.
- End-to-end: sign up/login → create workspace → create project → create task → move task → add comment → dashboard shows expected counts.

Definition of Done for every feature

- Acceptance criteria are met and edge cases considered.
- Backend permissions and validation are implemented, not just frontend checks.
- Tests are added/updated and pass.
- Linting, formatting and type checks pass.
- Loading, empty, success and error states are handled.
- API documentation and README are updated when behavior or setup changes.
- Secrets and private data are not present in the diff.
- Feature is reviewed via pull request and merged only after checks pass.

15. Security & Privacy Checklist

- Set DEBUG=False in production; use a strong SECRET_KEY from environment variables.
- Enforce HTTPS; set secure cookie flags when cookie-based authentication is used.

- Configure CORS with exact allowed origins; do not use wildcard origins with credentials.
- Choose and document a CSRF strategy when cookies are used; CORS is not a CSRF defense.
- Use least-privilege authorization and tenant-scoped querysets for every object.
- Validate that assignees, members and related objects belong to the same workspace.
- Apply reasonable throttling to login, registration and password reset endpoints.
- Keep dependencies patched and remove unused packages.
- Do not expose stack traces, database errors, secrets or token values in responses/logs.
- Use synthetic demo data and a dedicated demo account with no sensitive information.
- Review database backups/retention and provider limits; do not assume free-tier backup guarantees.

16. Risks & Mitigations

Risk	Mitigation
PythonAnywhere free-plan or Neon connectivity restrictions	Verify outbound connection to Neon, WSGI/Python support, package/static-file needs and current account limits before implementation; keep a compatible Python-host fallback documented. Add friendly loading/error states and disclose free-tier limitations.
Free plan limits change	Check current official pricing and plan restrictions for PythonAnywhere, Vercel and Neon before deployment; keep configuration portable through environment variables and document the tested deployment setup.
CORS/auth cookie configuration errors	Document origins, cookie flags and CSRF behavior; test production login from the deployed frontend.
Cross-tenant data leak	Centralize membership checks; use tenant-scoped querysets; write explicit negative permission tests.
Scope creep	Freeze MVP features; defer real-time WebSockets, email integrations, billing and advanced customization.
Frontend/backend work drifts	Define API contract early; use OpenAPI and shared TypeScript response types where practical.
Database migrations break deployment	Run migrations as a deliberate deploy step; test migrations against a staging/dev database first.
Demo feels empty	Seed fictional workspace, projects, tasks, comments and activity; provide safe demo credentials.
Single-person branch workflow becomes overhead	Use simple feature branches and PRs; omit develop branch if it adds no value.

17. Portfolio & GitHub Deliverables

- GitHub repository with a professional description, topics/tags, and a pinned release.
- README containing product overview, screenshots, live demo URL, demo login instructions, features, architecture, stack, local setup and environment variables.

- ERD diagram and concise architecture diagram under docs/.
- OpenAPI/API documentation or a Postman collection with example requests.
- Seed command or fixture for fictional demo data.
- Tests and GitHub Actions CI workflow.
- Deployment guide documenting Vercel, PythonAnywhere, Neon PostgreSQL, migrations, WSGI/static files, CORS/authentication and environment variables.
- At least 5–8 strong screenshots: landing, dashboard, project list, Kanban board, task details, members/settings, responsive view.
- A 60–90 second walkthrough video showing the core workflow.
- Upwork portfolio description focused on outcomes, not just a list of frameworks.

Suggested project description for portfolio

Nexus is a full-stack project management SaaS built with React, TypeScript, Django REST Framework and PostgreSQL. It supports workspace-based collaboration, role-based access, project organization, Kanban task workflows, comments, activity history and progress dashboards. The project emphasizes tenant isolation, API design, automated tests, responsive UX and documented deployment.

18. Recommended Build Order & Immediate Next Steps

25. Create the GitHub repository named nexus-project-management (or nexus-saas) and add a clear README.
26. Create frontend/ and backend/ folders; initialize React + Vite + TypeScript and Django + DRF.
27. Create the custom user model before the first database migration if a custom model is required.
28. Connect PostgreSQL locally and create initial apps: accounts, workspaces, projects, tasks, comments and activity.
29. Add health check, lint/format configuration and GitHub Actions early.
30. Implement auth and workspace membership before project/task APIs so authorization has a solid foundation.
31. Build the first vertical slice: sign in → create workspace → create project → create task.
32. Add Kanban interactions and persist status/order through the API.
33. Complete tests and seed data before polishing charts or optional notifications.
34. Deploy backend and database, then frontend; verify production auth, CORS, migrations and demo account.
35. Capture screenshots and record the walkthrough only after the live demo is stable.

Recommended MVP boundary: ship authentication, workspaces, projects, task list/Kanban, comments, activity, basic dashboard and deployment. Defer billing, advanced real-time collaboration, custom workflows and external integrations until the MVP is stable.